Configurar una zona DNS con BIND

Guía de entrenamiento: el ejercicio `carlitos.com.ar` explicado y todo lo que te pueden pedir en este tipo de consigna

72.07 Protocolos de Comunicación · ITBA · Todas las sintaxis de esta guía fueron probadas con BIND 9.18 (`named-checkzone` y `dig`)

Índice

1. El método: cómo atacar cualquier consigna de zona
2. El ejercicio carlitos.com.ar, explicado
3. Del enunciado al registro: la tabla para traducir frases
4. Los registros uno por uno, con la sintaxis textual
5. Tiempos y TTL
6. Nombres: el punto final, `@` y los relativos
7. Zona inversa (PTR), delegación y secundarios
8. Una zona completa con todas las variantes
9. Trampas: errores que avisa BIND y errores que NO avisa
10. Verificar con dig: qué consultar y qué mirar
11. Checklist para el parcial

1. El método: cómo atacar cualquier consigna de zona

Todas las consignas de DNS son la misma estructura con datos distintos. El orden que conviene seguir:

1. **Subrayar cada frase del enunciado** y asignarle un registro (tabla de la sección 3). Anotar los datos exactos: nombres, IPs, tiempos.
2. **Convertir los tiempos a segundos** antes de escribir nada.
3. **Mirar cómo está BIND**: `cat /etc/bind/named.conf.local` y `ls /etc/bind/`. Seguir el esquema que haya.
4. **Declarar la zona** en `named.conf.local` y **escribir el archivo** `db.<zona>`.
5. **Validar**: `named-checkzone`, `named-checkconf`.
6. **Aplicar**: `systemctl reload bind9`.
7. **Demostrar con dig** uno por uno cada pedido del enunciado.

> **Regla de oro:** cada frase del enunciado tiene que terminar siendo una línea del archivo y una consulta de `dig` que la muestre. Si una frase no tiene ninguna de las dos, te falta algo.

2. El ejercicio carlitos.com.ar, explicado

La consigna

Configurar un servidor DNS para servir la zona `carlitos.com.ar`:

- El servidor será la autoridad de dicha zona.
- El responsable debe ser `carloselmascapo@carlitos.com.ar`.
- `carlitos.com.ar` debe tener un registro A apuntando a `1.2.3.4` con un tiempo de vida de dos horas.
- El subdominio `mysite` debe ser un registro CNAME apuntando a `example.org` y debe tener una validez de una semana.
- El dominio para recibir correo electrónico debe ser `mail20`, `mail30` y `mail40.carlitos.com.ar` con las métricas 20, 30 y 40 respectivamente y una validez de 1 día.
- Muestre con dig cómo su servidor DNS sirve todos estos registros.

Cómo se lee cada frase

Frase	Qué hay que pensar	Resultado
"será la autoridad de dicha zona"	Autoridad = el servidor tiene la zona original. Bloque `zone` con `type master`, más un SOA y un NS en el archivo.	`type master;`
"El responsable debe ser ...@..."	Es el campo RNAME del SOA. La `@` se cambia por `.` y se agrega punto final.	`carloselmascapo.carlitos.com.ar.`
"registro A ... dos horas"	A en la raíz (`@`) con TTL explícito. 2 h = 7200 s.	`@ 7200 IN A 1.2.3.4`
"mysite ... CNAME ... example.org ... una semana"	CNAME con TTL explícito (604800). El destino es otro dominio: lleva punto final.	`mysite 604800 IN CNAME example.org.`
"dominio para recibir correo ... métricas 20, 30, 40 ... 1 día"	Tres MX en la raíz. "Métrica" = prioridad. 1 día = 86400.	`@ 86400 IN MX 20 mail20.carlitos.com.ar.` (y 30, 40)
"Muestre con dig"	Una consulta por tipo, siempre con `@127.0.0.1`.	`dig @127.0.0.1 ... +noall +answer`

El archivo final

```
$TTL    86400
@       IN      SOA     ns1.carlitos.com.ar. carloselmascapo.carlitos.com.ar. (
                              2026110701   ; Serial
                              3600         ; Refresh
                              1800         ; Retry
                              604800       ; Expire
                              86400 )      ; Minimum TTL

        IN      NS      ns1.carlitos.com.ar.
ns1     IN      A       1.2.3.4
@       7200    IN      A       1.2.3.4
mysite  604800  IN      CNAME   example.org.

@       86400   IN      MX      20      mail20.carlitos.com.ar.
@       86400   IN      MX      30      mail30.carlitos.com.ar.
@       86400   IN      MX      40      mail40.carlitos.com.ar.
mail20  IN      A       1.2.3.5
mail30  IN      A       1.2.3.6
mail40  IN      A       1.2.3.7
```

Los comandos, en orden

```
sudo nano /etc/bind/named.conf.local                                # agregar: zone
"carlitos.com.ar" { type master; file "..."; };
sudo nano /etc/bind/db.carlitos.com.ar                              # el archivo de arriba
sudo named-checkzone carlitos.com.ar /etc/bind/db.carlitos.com.ar   # valida la zona → "OK"
sudo named-checkconf                                                # valida la config → silencio
= OK
sudo systemctl reload bind9                                         # aplica sin cortar el
servicio
dig @127.0.0.1 carlitos.com.ar MX +noall +answer                    # demostrar
```

Los tres detalles que hacen la diferencia en este ejercicio:

1. La **@ del mail** pasa a punto en el SOA.
2. El **punto final** de `example.org.` : sin él queda `example.org.carlitos.com.ar.`
3. Los **TTL explícitos** van entre el nombre y `IN` , en segundos.

3. Del enunciado al registro: la tabla para traducir frases

Esta es la tabla más útil para entrenarte: leé la columna de la izquierda como si fuera la consigna y asegurate de saber escribir la de la derecha sin mirar.

El enunciado dice...	Registro	Línea de ejemplo
"será la autoridad", "servir la zona"	zone + SOA + NS	`zone "x.com" { type master; file "..."; };`
"responsable", "contacto", "administrador ... @"	SOA (RNAME)	`admin.x.com.` (la @ pasa a punto)
"tenga una IP", "apunte a una dirección IPv4"	A	`@ IN A 1.2.3.4`
"IPv6", "dirección IPv6"	AAAA	`@ IN AAAA 2001:db8::10`
"alias", "apunte al nombre de", "mismo sitio que", "www debe apuntar a la raíz"	CNAME	`www IN CNAME @` o `www IN CNAME otro.com.`
"recibir correo", "servidor de correo", "métrica / prioridad / preferencia"	MX	`@ IN MX 10 mail.x.com.`
"mismos servidores de correo que el dominio Y (manteniendo el orden)"	MX copiados	consultar `dig Y MX` y copiar con las mismas prioridades
"solo la IP X puede enviar correo", "SPF", "política de envío"	TXT (SPF)	`@ IN TXT "v=spf1 ip4:1.2.3.4 -all"`
"DKIM", "DMARC"	TXT	`_dmarc IN TXT "v=DMARC1; p=reject"`
"validez", "tiempo de vida", "TTL", "caché por"	TTL en esa línea	`@ 7200 IN A ...`
"tiempo que se recuerda que un nombre no existe"	Minimum del SOA	último número del SOA
"servidor de nombres", "otro servidor DNS", "delegar el subdominio"	NS	`ventas IN NS ns.ventas` + A del NS
"cualquier subdominio", "cualquier nombre"	comodín	`* IN A 10.0.0.99`
"dos IPs", "balanceo", "varias direcciones"	varios A	dos líneas A con el mismo nombre
"zona inversa", "dada la IP, resolver el nombre", "reverse"	PTR en `x.x.x.in-addr.arpa`	`10 IN PTR www.x.com.`
"servicio SIP / XMPP / LDAP", "puerto del servicio"	SRV	`_sip._tcp IN SRV 10 60 5060 sip`
"qué autoridad puede emitir certificados"	CAA	`@ IN CAA 0 issue "letsencrypt.org"`
"que los cambios se propaguen", "secundario"	serial + slave	subir el serial; `type slave; masters {IP;};`

4. Los registros uno por uno

Todas estas líneas se probaron en una zona `ejemplo.com.ar` y `dig` las sirvió como se muestra.

SOA, NS, A, AAAA

```
@   IN SOA  ns1 juan\.perez.ejemplo.com.ar. ( 2026110701 1h 30m 1w 1d )
    IN NS   ns1
ns1 IN A    10.0.0.1
@   2h IN A 10.0.0.10
@   IN AAAA 2001:db8::10
```

- El SOA lleva: servidor primario, responsable, y cinco números (serial, refresh, retry, expire, minimum).
- Si el responsable tiene un punto antes de la @ (`juan.perez@...`) se escapa: `juan\.perez.ejemplo.com.ar.` dig lo muestra igual con la barra.
- Una línea que empieza con espacios hereda el nombre de la anterior (por eso el NS es de `@`).

CNAME (alias)

```
www   1w IN CNAME @                 # alias de la raíz
blog     IN CNAME ejemplo.com.ar.   # alias con nombre completo
mysite   IN CNAME example.org.      # alias a otro dominio
```

```
$ dig @127.0.0.1 www.ejemplo.com.ar A +noall +answer
www.ejemplo.com.ar.  604800  IN  CNAME  ejemplo.com.ar.
ejemplo.com.ar.      7200    IN  A      10.0.0.10
```

> **Reglas del CNAME:** un nombre con CNAME **no puede tener ningún otro registro** (BIND: `CNAME and other data`). Y la raíz de la zona (`@`) no puede ser CNAME, porque ya tiene SOA y NS.

MX (correo)

```
@  IN MX 10 mx1                     # servidor dentro de la zona (relativo)
@  IN MX 20 mx2.otroproveedor.net.  # servidor de otro dominio (absoluto)
mx1 IN A 10.0.0.25                  # el servidor tiene que resolver
```

```
$ dig @127.0.0.1 ejemplo.com.ar MX +noall +answer
ejemplo.com.ar.  86400  IN  MX  10 mx1.ejemplo.com.ar.
ejemplo.com.ar.  86400  IN  MX  20 mx2.otroproveedor.net.
```

- **Menor número = más prioridad.** Se intenta primero el 10, y el 20 es el respaldo.
- Un MX apunta a un **nombre**, nunca a una IP, y ese nombre no puede ser un CNAME (BIND: `is a CNAME (illegal)`).
- Si el destino está en tu zona, agregale su registro A.

TXT: SPF, DMARC, DKIM

```
@               IN TXT "v=spf1 ip4:1.2.3.4 -all"
_dmarc          IN TXT "v=DMARC1; p=reject; rua=mailto:dmarc@ejemplo.com.ar"
sel1._domainkey IN TXT ( "v=DKIM1; k=rsa; p=AAAA"
                         "BBBB" )
```

SPF (en el TXT de la raíz)	Significa
`v=spf1`	Es obligatorio, va primero.
`ip4:1.2.3.4` / `ip6:...`	Esa IP puede enviar. También acepta un rango: `ip4:10.0.0.0/24`.
`mx` / `a`	Pueden enviar los servidores MX / la IP del A de la zona.
`include:otro.com`	Se suman los permitidos de otro dominio.
`-all` / `~all`	El resto: rechazar (-) o sospechoso (~). "Solo X puede enviar" = `-all`.

Comillas obligatorias. Probado: sin comillas, `named-checkzone` dice `OK` pero guarda tres strings separados (`"v=spf1" "ip4:1.2.3.4" "-all"`) y el SPF queda roto. Un TXT largo se parte en varias comillas dentro de paréntesis, como el DKIM de arriba.

Comodín y varios A

```
*   IN A 10.0.0.99      # cualquier nombre que no exista responde esta IP
ftp IN A 10.0.0.21
    IN A 10.0.0.22      # mismo nombre, dos IPs: dig devuelve las dos
```

Probado: `cualquiera.ejemplo.com.ar` resolvió a `10.0.0.99`, y `ftp` devolvió las dos IPs.

SRV y CAA

```
_sip._tcp IN SRV 10 60 5060 sip        # prioridad  peso  puerto  servidor
sip       IN A   10.0.0.50
@         IN CAA 0 issue "letsencrypt.org"
```

```
_sip._tcp.ejemplo.com.ar.  86400  IN  SRV  10 60 5060 sip.ejemplo.com.ar.
```

El nombre SRV tiene el formato `_servicio._protocolo`. Los dos primeros números son prioridad y peso (para repartir carga); después el puerto y el servidor.

5. Tiempos y TTL

Pedido	Cuenta	Segundos	Forma abreviada
30 minutos	30 × 60	`1800`	`30m`
1 hora	60 × 60	`3600`	`1h`
2 horas	2 × 3600	`7200`	`2h`
12 horas	12 × 3600	`43200`	`12h`
1 día	24 × 3600	`86400`	`1d`
1 semana	7 × 86400	`604800`	`1w`
2 semanas	14 × 86400	`1209600`	`2w`

BIND acepta las abreviaturas `s m h d w` (probado: `2h` y `1w` se sirvieron como `7200` y `604800`). En el parcial conviene escribir **segundos**, porque es lo que muestra `dig` y así comparás directo.

Cómo se aplica el TTL

- `$TTL` al principio: valor por defecto para todo lo que no tenga uno propio.
- TTL en la línea (entre nombre e `IN`): pisa al `$TTL` solo para ese registro.
- El último número del SOA no es el TTL del SOA: es el TTL del **caché negativo** (cuánto se recuerda que algo no existe).
- Los registros de un mismo nombre y tipo (por ejemplo los tres MX) deben tener el mismo TTL. Si ponés distintos, BIND avisa `TTL set to prior TTL` y usa el del primero (probado), así que el TTL que escribiste en los otros se pierde.

6. Nombres: el punto final, @ y los relativos

Escribís	BIND lo interpreta como	Cuándo
`@`	`carlitos.com.ar.` (la zona)	La raíz de la zona.
`mysite` (sin punto)	`mysite.carlitos.com.ar.`	Nombre dentro de la zona.
`example.org.` (con punto)	`example.org.` tal cual	Nombre de otro dominio, o completo.
`example.org` (sin punto)	`example.org.carlitos.com.ar.`	**Error típico.**

El error que BIND no avisa. Probado: quitando el punto final de `mx2.otroproveedor.net.`, `named-checkzone` dice `OK` y el MX queda como `mx2.otroproveedor.net.ejemplo.com.ar.` Siempre revisá con `dig` que el destino sea el que querías.
Regla: nombre de *esta* zona → sin punto o con el nombre completo y punto; nombre de *otra* zona → **siempre con punto final**.

7. Zona inversa, delegación y secundarios

Zona inversa (PTR)

Dada una IP, devuelve un nombre. Para la red `10.0.0.0/24` la zona se llama **los octetos de la red al revés** + `.in-addr.arpa`:

```
# named.conf.local
zone "0.0.10.in-addr.arpa" { type master; file "/etc/bind/db.10.0.0"; };

# db.10.0.0
$TTL 1d
@   IN SOA ns1.ejemplo.com.ar. admin.ejemplo.com.ar. ( 2026110701 1h 30m 1w 1d )
    IN NS  ns1.ejemplo.com.ar.
10  IN PTR www.ejemplo.com.ar.      # el 10 es el último octeto de 10.0.0.10
25  IN PTR mx1.ejemplo.com.ar.
```

```
$ dig @127.0.0.1 -x 10.0.0.10 +noall +answer
10.0.0.10.in-addr.arpa.  86400  IN  PTR  www.ejemplo.com.ar.
```

- Red `192.168.5.0/24` → zona `5.168.192.in-addr.arpa`.
- El nombre del PTR siempre con punto final.
- `dig -x IP` arma la consulta inversa por vos.

Delegar un subdominio

"El subdominio `ventas` lo administra otro servidor": se pone un NS para `ventas` y la IP de ese servidor (registro *glue*, necesario cuando el NS está dentro del subdominio delegado):

```
ventas    IN NS ns.ventas
ns.ventas IN A  10.0.1.1
```

```
$ dig @127.0.0.1 host.ventas.ejemplo.com.ar A +noall +authority +additional
ventas.ejemplo.com.ar.     86400 IN NS ns.ventas.ejemplo.com.ar.
ns.ventas.ejemplo.com.ar.  86400 IN A  10.0.1.1
```

El servidor ya no responde por `host.ventas...`: **deriva** la consulta al NS delegado. `named-checkzone` imprime un aviso sobre el glue, pero carga la zona y funciona.

Servidor secundario (slave)

```
# En el SECUNDARIO
zone "carlitos.com.ar" { type slave; masters { 192.168.1.10; }; file "db.carlitos.com.ar"; };

# En el PRIMARIO (permitir que el secundario copie la zona)
zone "carlitos.com.ar" { type master; file "..."; allow-transfer { 192.168.1.20; }; also-notify {
192.168.1.20; }; };
```

El secundario copia la zona solo si el **serial subió**. Por eso, cada vez que modificás el archivo: **incrementá el serial** (`2026110701` → `2026110702`) y recargá con `sudo rndc reload carlitos.com.ar`.

8. Una zona completa con todas las variantes

Este archivo (probado, `named-checkzone` → OK) reúne todo lo anterior. Sirve para ver cómo conviven los registros:

```
$TTL 1d
$ORIGIN ejemplo.com.ar.
@         IN SOA  ns1 juan\.perez.ejemplo.com.ar. ( 2026110701 1h 30m 1w 1d )
          IN NS   ns1
          IN NS   ns2.otroproveedor.net.
ns1       IN A    10.0.0.1
@    2h   IN A    10.0.0.10
@         IN AAAA 2001:db8::10
www  1w   IN CNAME @
blog      IN CNAME ejemplo.com.ar.
@         IN MX 10 mx1
@         IN MX 20 mx2.otroproveedor.net.
mx1       IN A    10.0.0.25
@         IN TXT  "v=spf1 ip4:1.2.3.4 -all"
_dmarc    IN TXT  "v=DMARC1; p=reject; rua=mailto:dmarc@ejemplo.com.ar"
*         IN A    10.0.0.99
_sip._tcp IN SRV  10 60 5060 sip
sip       IN A    10.0.0.50
@         IN CAA  0 issue "letsencrypt.org"
ventas    IN NS   ns.ventas
ns.ventas IN A    10.0.1.1
ftp       IN A    10.0.0.21
          IN A    10.0.0.22
```

`$ORIGIN` fija el sufijo para los nombres relativos. Si no lo ponés, BIND usa el nombre de la zona del `named.conf`, que es lo habitual.

9. Trampas: errores que avisa BIND y errores que NO avisa

Los que avisa (mensajes reales)

Mensaje	Causa	Arreglo
`unknown RR type 'CNAM'`	Tipo mal escrito.	Corregirlo (la línea viene en el mensaje).
`CNAME and other data`	Un nombre con CNAME y otro registro.	Dejar solo el CNAME, o sacarlo.
`NS '...' has no address records`	El NS está en la zona y no tiene A.	Agregar `ns1 IN A ip`.
`has no NS records`	Falta el NS de la zona.	Agregar `IN NS ns1`.
`no owner`	La primera línea es un registro sin nombre (falta el SOA o está mal).	Empezar con el SOA.
`MX '...' is a CNAME (illegal)`	MX apuntando a un alias.	Apuntar a un nombre con A.
`missing ';' before '}'` (named-checkconf)	Falta un `;`.	Agregarlo.

Los que NO avisa (los peligrosos: dan OK y están mal)

Error	Qué pasa	Cómo lo detectás
Falta el punto final en un destino externo	`example.org` → `example.org.carlitos.com.ar.`	`dig` y mirar el destino.
TXT sin comillas	Se parte en varios strings; el SPF no funciona.	`dig TXT`: tiene que ser un solo string entre comillas.
Serial sin subir	Los secundarios no se enteran; algunos clientes siguen con lo viejo.	Comparar el serial del SOA.
@ del mail en el SOA	Cambia el significado de la línea.	Revisar el RNAME en `dig SOA`.
Nombre de zona distinto en `named.conf.local` y en `named-checkzone`	Valida una cosa y sirve otra.	Que el nombre sea idéntico.
Olvidar el `reload`	`dig` muestra la versión anterior.	Recargar siempre después de editar.

10. Verificar con dig: qué consultar y qué mirar

```
dig @127.0.0.1 NOMBRE TIPO                  # salida completa
dig @127.0.0.1 NOMBRE TIPO +noall +answer   # solo la respuesta (limpio para mostrar)
dig @127.0.0.1 NOMBRE TIPO +short           # solo el dato
dig @127.0.0.1 -x 10.0.0.10 +noall +answer  # consulta inversa
dig @127.0.0.1 ZONA AXFR                    # toda la zona de una vez
```

- `@127.0.0.1` es imprescindible: asegura que respondió *tu* BIND y no el resolvedor del sistema.
- `status: NOERROR` = existe; `NXDOMAIN` = ese nombre no existe; `REFUSED` = este servidor no tiene esa zona; `SERVFAIL` = la zona no cargó (mirá `journalctl -u named`).
- `flags: aa` = respuesta autoritativa: la prueba de que el servidor es la autoridad.
- La columna del medio es el **TTL**: tiene que coincidir con lo que pidió el enunciado.
- Una consulta por cada pedido del enunciado: si piden A, CNAME y MX, tres consultas.

Para demostrar...	Comando
Registro A	`dig @127.0.0.1 carlitos.com.ar A +noall +answer`
Un CNAME	`dig @127.0.0.1 mysite.carlitos.com.ar CNAME +noall +answer`
MX (con prioridades)	`dig @127.0.0.1 carlitos.com.ar MX +noall +answer`
Responsable y datos de la zona	`dig @127.0.0.1 carlitos.com.ar SOA +noall +answer`
SPF	`dig @127.0.0.1 carlitos.com.ar TXT +noall +answer`
Que sirve como autoridad	`dig @127.0.0.1 carlitos.com.ar SOA` y buscar `aa`
Todo junto	`dig @127.0.0.1 carlitos.com.ar AXFR`

11. Checklist para el parcial

1. ☐ Subrayé cada frase del enunciado y le asigné un registro.
2. ☐ Convertí todos los tiempos a segundos.
3. ☐ Miré `named.conf.local` y `ls /etc/bind/` antes de tocar nada.
4. ☐ Declaré la zona (`type master` si "es la autoridad") y el nombre coincide en todos lados.
5. ☐ El SOA tiene el responsable con la `@` convertida en punto.
6. ☐ Hay al menos un NS, y si está dentro de la zona, tiene su A.
7. ☐ Los destinos de otros dominios (CNAME, MX, NS externos) llevan **punto final**.
8. ☐ Los TXT van entre comillas.
9. ☐ Cada MX apunta a un nombre que resuelve y no es un CNAME.
10. ☐ `named-checkzone` → OK y `named-checkconf` → silencio.
11. ☐ `systemctl reload bind9` y `status` con `active (running)`.
12. ☐ Demostré con `dig @127.0.0.1` cada pedido del enunciado, mirando TTL, prioridad y destino.
13. ☐ Si cambié algo después: subí el serial y recargué.

Resolución paso a paso del ejercicio con las salidas reales: `dns/zona-carlitos-com-ar_repaso.md`